The AI Is Fast.

But You're the Engineer.

Specs, Review, and Toolchains for AI-Assisted Development

Satyam Mishra | AI Engineer, HROne

THE STORY

I built an agentic RAG system in 20 minutes.

It hallucinated policy details that didn't exist in any document.

Generated pipeline	20 minutes with Claude Code
Code ran, answers looked right	Happy path worked
Tested with real query	Agent skipped retrieval
Hallucinated leave policy	Confidently wrong at scale

Speed without structure just means you ship mistakes faster.

70%

of the way there

AI coding tools get you to working software, fast.

The code runs. The happy path works. You feel almost done.

You are not almost done.

The 30% gap:

Your data. Your failure modes. Your system.

Context is everything in production.

REAL EXAMPLE

Vector search returned wrong chunks in production

What the AI assumed

- Clean, well-formed queries
- Standard English text input
- Embeddings match on semantics
- Default chunk size is fine

What actually happened

- Abbreviations, shorthand, slang
- Mixed Hindi and English
- Embedding model not trained for this
- Wrong chunks returned consistently

The AI that wrote the retrieval code had no idea how real employees search.

THE DIFFERENCE

Same AI. Same model. Same task.

✗ Typical Prompt

"Build a RAG pipeline for HR policy docs"

Generic LangChain chain

Default chunk size

No reranking

No confidence fallback

Breaks on real queries

✓ Proper Spec

Your embedding model + vector DB

Your document types

Actual user query examples

Confidence threshold behavior

Modules to reuse

What NOT to do

The spec took **10 minutes**. It saved **2 hours**.

Two Skills and a Toolchain

Keep the speed. Trust the output.

01

Spec-Driven Development

Tell the AI exactly what you need

02

Adversarial Review

Find where the AI is wrong

03

The Toolchain

Systems that make it repeatable

SKILL ONE

Spec-Driven Development

Your spec is your highest-leverage artifact.

The Anatomy of a Great Spec

Six components. Ten minutes to write. Hours saved.

01

Context

Tech stack, relevant files, and WHY this feature exists. Not just 'what to build' but 'what problem it solves.'

02

Requirements + Edge Cases

Not just 'retrieve relevant documents.' But 'what happens when no documents match? What if the query is in Hindi?'

03

Non-functional Targets

Not 'it should be fast.' Say 'response under 2 seconds at P95.' Numbers, not vibes.

The Anatomy of a Great Spec

04 **Examples**

Actual input queries and expected output. The AI performs dramatically better with concrete examples.

05 **Anti-requirements**

What NOT to do. 'Don't add new pip packages.' 'Don't modify the agent orchestrator.' This prevents the most problems.

06 **Definition of Done**

What tests must pass. What the output should look like in production. No ambiguity.

I spend **30-40%** of my AI-assisted coding time writing specs. **That's where all the leverage is.**

SKILL TWO

Adversarial Review

Don't review to confirm it works. Review to find where it's wrong.

The 5-Point Review Checklist

The AI is a fast junior developer. You are the senior reviewer.

1. **Logic**
   Take one real input, trace it through the code manually.
2. **Edge Cases**
   Empty query? Wrong language? Document longer than chunk size?
3. **Failure Handling**
   Vector DB slow? LLM API timeout? Zero retrieval results?
4. **Dependencies**
   What packages were added? Needed? Maintained?
5. **Consistency**
   Does this match how the rest of your system works?

THE TOOLCHAIN

Systems That Make It Repeatable

Specs and review are skills. This is the infrastructure.

TOOLCHAIN 01

Project Config File

Every AI session starts with your context already loaded.

Claude Code	Cursor	Codex
CLAUDE.md	**.cursor-rules**	**AGENTS.md**

Stack: Python, FastAPI, LangChain, MongoDB

Conventions: Type hints, docstrings, async functions

Never: Add new pip packages without asking

Never: Modify the agent orchestrator directly

Prefer: MongoDB aggregation over client-side filtering

TOOLCHAIN 02

Slash Commands

Reusable prompt templates. Write once, invoke with a slash.

/test-rag-pipeline — 15 min to create

Runs a query through the retrieval pipeline, checks embedding scores, verifies chunks, evaluates response quality.

/optimize-mongo-query — 15 min to create

Takes a file path, analyzes for missing indexes, inefficient aggregation stages, timeout risks with large collections.

The filename becomes the command name:

```
.claude/commands/optimize-mongo-query.md
```

TOOLCHAIN 03

Skills Folders

Bundles of instructions, scripts, and reference files. Auto-detected by context.

api-reviewer

- SKILL.md with activation rules
- Auth + error handling scripts
- API design standards docs

Activates when building/modifying an API endpoint

deploy-checker

- Environment variable validation
- Database connection tests
- Production readiness checks

Activates before shipping to production

Build them once. They get smarter as you add to them.

TOOLCHAIN 04

Parallel Worktrees

Multiple AI sessions in parallel. No conflicts.

Terminal 1

```
$ git worktree
feature/ess-payslip
```

ESS payslip feature

AI session active

Terminal 2

```
$ git worktree
feature/rag-optimize
```

RAG pipeline optimization

AI session active

Terminal 3

```
$ git worktree
refactor/agent-orchestrator
```

Agent orchestrator refactor

AI session active

In each session: config loads automatically, skills are available, slash commands work.

BONUS

Playwright MCP

Give your AI coding tool actual browser eyes.

/test-UI localhost:3000 > Opens browser, waits > Takes screenshot > Checks console errors > Reports back

One line to set up. No more switching between terminal and browser.

Five Things You Can Do This Week

01	**Create a project config file** CLAUDE.md / .cursor-rules / AGENTS.md	30 min
02	**Write one proper spec** All six components before your next AI session	15 min
03	**Run the 5-item review checklist** Logic, edge cases, failure, dependencies, consistency	20 min
04	**Create one slash command** For a task you repeat weekly. One markdown file.	5 min
05	**Try parallel worktrees** Two terminals, two branches, two AI sessions	10 min

RESOURCES

Download the Full Guide

Scan to download. Everything from this talk, plus more.

The Complete Guide

11 sections, specs to ship

Hands-On Assignment

Build a release notes generator

Speed + **Direction** + **Confidence** + **Consistency**

AI | Specs | Review | Toolchain

The AI Is Fast.
But You're the Engineer.

Satyam Mishra

AI Engineer, HROne | Building AI systems